Algorithms, 2026 Fall, Homework 3
(Due: Oct 8)

September 28, 2026

Required Readings: Textbook Ch 4.3, 4.7

You can discuss homeworks with your classmates or using LLMs. But you will have to write down the solution on your own. You need to acknowledge other students who help you in the homework. If you read some other source that is helpful, you should list all of them.

Problems marked with (@) can be skipped by those who have extensive experience in OI.

Problem 1 : (@) Implement the heapsort algorithm using an array A of length n. Suppose the initial sequence is stored in A. You are allowed to use $O(1)$ extra working space, and the output sequence should be stored in A as well. In other words, you can not open another large array to store intermediate results. Such an algorithm is called an in-place algorithm. Write down the pseudo-code for it.

Problem 2 : (@) Try to implement Dijkstra's algorithm using priority queue. Your algorithm should run in time $O(|E| \log |V|)$. Write down the pseudo-code for it.

Problem 3 : (@) You are given a directed graph with n vertices and m edges. You need to answer many queries of the form:

"Is vertex u able to reach vertex v"?

A naive BFS/DFS per query is too slow.

Instead, we would like to find a better strategy leveraging the SCCs of the graph: Build the meta-graph we mentioned in the class; then answer queries efficiently using the meta-graph.

Write down the pseudo-code for all details. What is the time complexity of your algorithm (time for building the reachability data structure, and the time for answering the query – which can depend on the size of the meta-graph)

Problem 4 : (@) KT book pp 191 problem 6

Problem 5 : KT book pp 193 problem 11

Problem 6 : KT book pp 192 problem 9

Problem 7 : KT book pp 197 problem 17

Problem 8 : KT book pp 203 problem 29

Problem 9 : In the algorithm for finding strongly connected component, what happens if we modify the algorithm in the following way: We do not compute the transpose of G; instead, we perform another round of DFS, but the outer loop uses the increasing order of the finishing times of all nodes. Is this algorithm correct? Prove its correctness or provide a counter example.